

9位解法

Rank	9
Team	Vibes and Genius Trade-Off
Author	Tom
Topic ID	611908
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/rsna-intracranial-aneurysm-detection/discussion/611908>

Retrieved with Kaggle CLI on 2026-07-07.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

Acknowledgement

competition hostの@evancalabrese、@shosys、data preparationとcompetition processに関わったRSNAの皆様、competitionを企画したKaggle staffに感謝します。

team **Vibes and Genius Trade-Off** (@tom99763、@sersasj、@iamparadox、@chihantsai、@atom1231) の解法を紹介します。

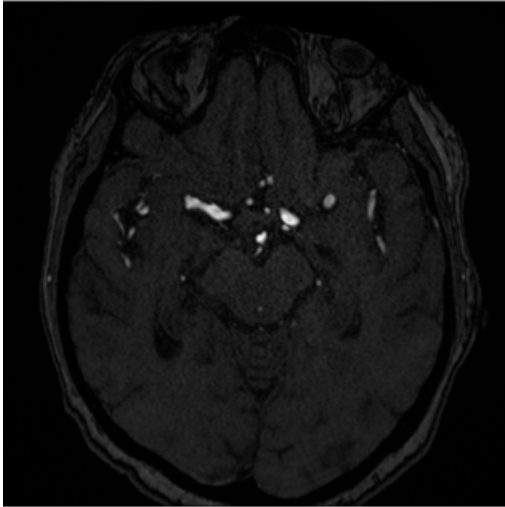
TL;DR

3つの相補的なapproachを組み合わせた。

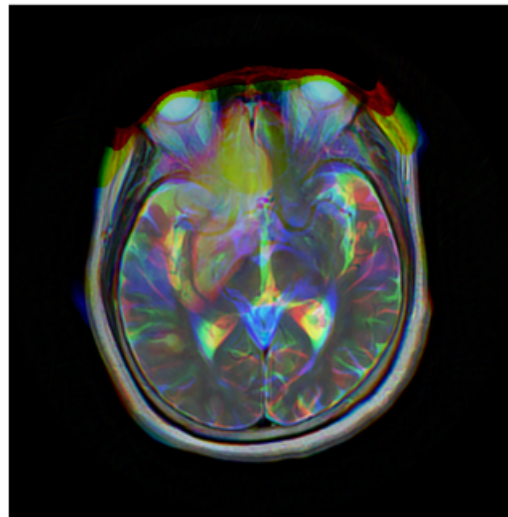
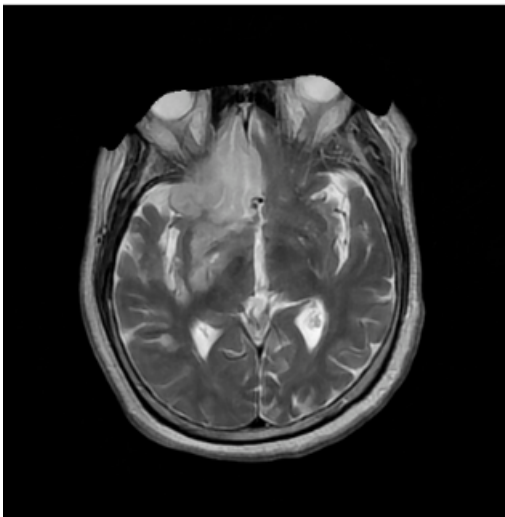
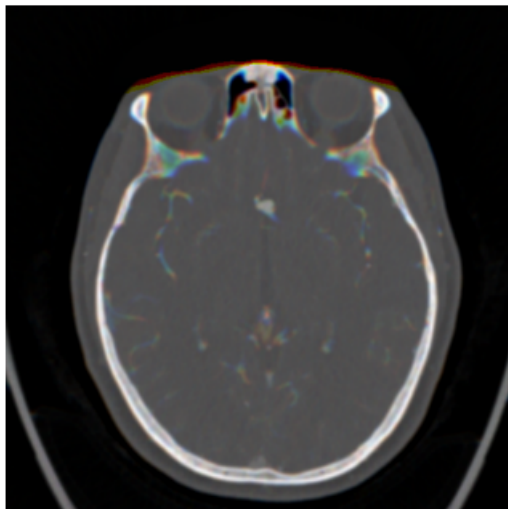
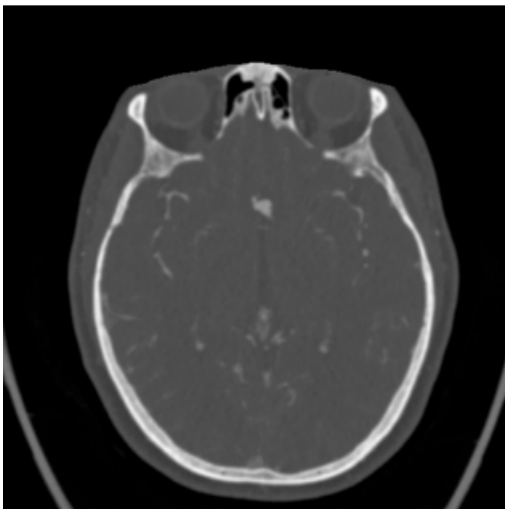
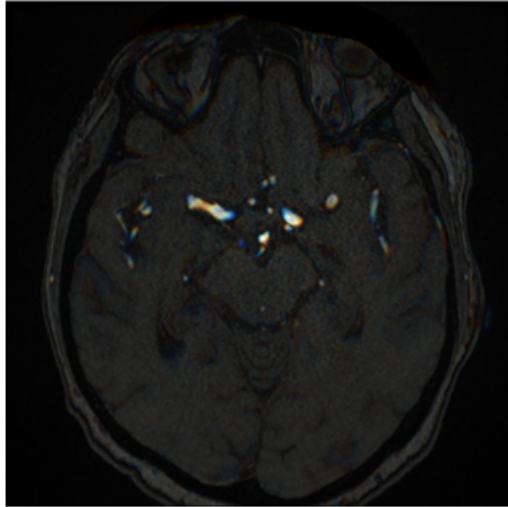
1. 異なるbackboneを持つYOLO 2.5D。 [BYU competition](#)から派生。
2. 2D EffV2s extractorを持つ3D CenterNet。
3. Meta-classifier (LightGBM、XGBoost、CatBoost) 。

YOLO 2.5D model、2D EffV2s extractor付き3D CenterNet、3つのmeta-classifierのoutputを平均してfinal probabilityを得た。

2D



2.5D



YOLO 2.5D

YOLOは2 variantを使った。

- **YOLOv11m**: Ultralyticsのstandard YOLO11 medium model。
- **Custom YOLO with timm backbone**: backboneに `timmm/tf_efficientnetv2_s.in21k_ft_in1k` を持つ YOLO architecture。customizationの詳細は[BYU write-up](#)を参照。

13 vessel locationを別々のbounding-box classとして扱った。

- Left/Right Infraclinoid Internal Carotid Artery
- Left/Right Supraclinoid Internal Carotid Artery
- Left/Right Middle Cerebral Artery
- Anterior Communicating Artery
- Left/Right Anterior Cerebral Artery
- Left/Right Posterior Communicating Artery
- Basilar Tip
- Other Posterior Circulation

2.5D strategy

imageを512×512へresizeし、min-max normalization後に次の2.5D形式へ変換した。

```
R (Red channel)  = slice i-1
G (Green channel) = slice i
B (Blue channel) = slice i+1
```

例を見ると分かるように、**Z spacingをstandardizeしなかった**ためimageは大きく異なる。[@sersasj](#)はZ-axis resizeを1〜2週間実験したが、結果は一貫して悪かった。何かを間違えていた可能性はあるが、追加調査の時間はなかった。proper Z-spacing resizeなしの2.5Dには非常に消極的だったが、それでもZ-resamplingなしの2.5DでCVが0.02超改善した。

Training configuration

Data sampling:

- negative sampleではseriesごとに等間隔の10 sliceを使った。100-slice seriesなら `[1, 11, 22, 33, 44, 56, 67, 78, 89, 99]`。
- positive sampleではannotationを含む全sliceを使った。

YOLO+timmm/EfficientNetV2-S backboneとYOLOv11mを学習した。fitness functionにはAUC metricも加え、mAP@50を優先した。

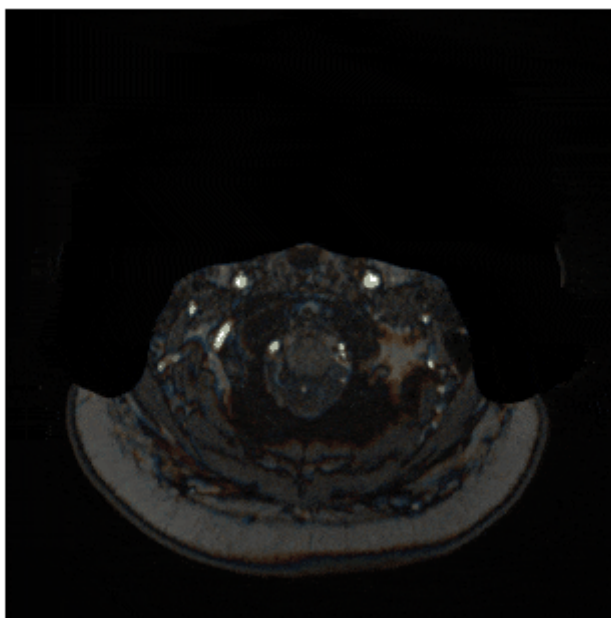
```
fitness =  × mAP@ +  × mAP@- +  × mAUC
```

効果は少しあったかもしれない。BYU competitionではUltralytics標準の $1.0 \times \text{mAP}@50-95$ より $\text{mAP}@50$ を優先した方が良い結果だった。AUCはcompetition metricなので加えるのが良いと思ったが、performanceへの大きな寄与は十分調べていない。benefitはわずかだった可能性もあるが、consistencyのため保持した。

User	GPU	CPU	RAM	Time/fold	Total (5 folds)
@sersasj	RTX 3090	Intel Core i5-12400F (12) @ 4.4GHz	32GB	4-5 h	~24 h
@iamparadox	RTX 4090	AMD Ryzen 9 7950X (32) @ 5.883GHz	64GB	~2 h	~10 h

Inference

DICOM sliceをspatial position (`SliceLocation` → `ImagePositionPatient` → `InstanceNumber`) でsortし、slice 1からN-1まで2.5D tripletを作る。両modelでbatch inferenceし、classごとのmax confidenceを抽出する。`max(localization_confidences)` をoverall aneurysm presence probabilityとして使った。



Cross-validation scores

`MultilabelStratifiedKFold` を使い、Aneurysm presence、13 vessel locationすべて、Modalityの分布がbalancedになるようfoldをstratifyした。

YOLO11m 2.5D:

Fold	loc_macro_auc	cls_auc	combined_mean
fold0	0.8184	0.7549	0.7867
fold1	0.8114	0.7897	0.8005
fold2	0.8134	0.7827	0.7981
fold3	0.8162	0.7872	0.8017
fold4	0.8540	0.8281	0.8410

Fold	loc_macro_auc	cls_auc	combined_mean
Average	0.8227	0.7885	0.8056

EfficientNetV2-S 2.5D:

Fold	loc_macro_auc	cls_auc	combined_mean
fold0	0.7978	0.7790	0.7884
fold1	0.8159	0.8269	0.8214
fold2	0.8155	0.7921	0.8038
fold3	0.8153	0.7907	0.8030
fold4	0.8499	0.8560	0.8529
Average	0.8189	0.8090	0.8139

Ensemble:

Fold	loc_macro_auc	cls_auc	combined_mean
fold0	0.8393	0.7849	0.8121
fold1	0.8330	0.8291	0.8310
fold2	0.8399	0.8135	0.8267
fold3	0.8370	0.8093	0.8232
fold4	0.8738	0.8620	0.8679
Average	0.8446	0.8198	0.8322

うまくいかなかったもの

- YOLO開発初期、@sersasjは3xYOLO11mと[LB 0.69のEfficientNetB2 public notebook](#)をensembleし、LB 0.78でcompetition序盤のtop 3へ入った。LBを少しprobeした後、@iamparadoxはYOLOのaneurysm classification AUROCが約0.58と非常に低いことに気づいた。そこでYOLOを補完しclassification AUROCを高めるmodelを開発した（後に初期YOLOの低性能はnegative sample不足が原因だと判明）。
- auxiliary segmentation headを持ち、series内のaneurysm presenceだけをdetectする2.5D modelも使った。[このnotebook](#)から着想を得ており、YOLO11mとの組み合わせでLB 0.80だった。その後@chihantsai、@atom1231とのmergeで不要になった。
- BYUではtimm/convnextv2_base.fcmae_ft_in22k_in1kがbest backboneだったが、今回はhalf precisionでlossがすぐNaNになった。ConvNeXtに共通する問題と思われる。full precisionならYOLO11mやtimm/tf_efficientnetv2_s.in21k_ft_in1kに近い性能だった可能性があるが、時間制約で検証できなかった。
- neck/headの変更も検討した。@tatamikennの[reverse-engineering notebook](#)からP3 featureが最も使われているように見えた。single foldで短く試したがvalidationはほぼ同じで、先へ進んだ。
- [3D segmentation result](#)を使ってYOLOのinvalid location predictionをfilterしたが、recallが大幅に低下した。

- YOLOでpatchを抽出し、wavelet/log-polar transform後にclassを予測する[approach](#)も良い結果ではなかった。

EfficientV2s + 3D-CenterNet (Flayer)

AIの助けを多く借りた非常にexperimentalなprojectである。人気の「LB 0.69 Public Notebook」から出発し、training pipelineとdata preprocessing全体を再構築して、aneurysm centerをdetectするCenterNet-like mechanismを組み込んだ。

Model architecture

slice-wise 2D encoder + shallow 3D head設計を採用した。

- **Encoder:** timmの `features_only` modeでImageNet-pretrained EfficientNetV2-S (`tf_efficientnetv2_s.in21k_ft_in1k`) を使い、penultimate feature map (`out_indices = (1, -2)`) を取得。
- **Auxiliary 2D head:** intermediate feature map (early stage) を軽量2D convolutional headへ渡し、slice単位のauxiliary logitsを出す。3D aggregation前にencoderがvessel-relevantなspatial cueを捉えるよう促し、slice-level targetでsuperviseしてearly-layer learningを安定させる。
- **2D→3D fusion:** 各volume (`B, C=1, D, H, W`) を `B·D` 枚の2D sliceへrearrangeし、個別にencodeした後、3D head前で (`B, C_feat, D, H', W'`) へ再構成する。backboneが3 channelを要求する場合、single-channel inputをchannel repeatする。
- **3D temporal head:** 軽量head (`Conv3d-BN-ReLU × 2, base_channels`) でdepth方向のsignalをaggregate。
- **Outputs:** 2つの `1×1×1` conv head。Heatmap headは `num_classes = 13` のvessel-location 3D centerness map、Offset headは3-channel sub-voxel offset (`dz, dy, dx`)。

Supervision targets

- class-specific heatmap上の各annotated centerへstride-awareな3D Gaussian peakを置く。
- nearest grid cellにsub-voxel residual offsetを保存し、offset maskで印を付ける。
- original series sizeからcurrent (`D, H, W`) へcoordinateをscaleし、(`stride_d=1, stride_h=stride_w=16`) でoutput gridへmapする。outputは `D × H/16 × W/16`。
- default Gaussian $\sigma = 0.2$ 。

Segmentation masks

volumeを0.7 mm isotropic spacingへresampleし、3D DynUNetを学習してphysical spaceのvessel maskを生成した。提供された13 vessel classのvoxel-level annotationをsingle binary vessel maskへmergeして学習した。得られた3D maskをslice-wise 2D maskへ変換し、auxiliary headをsuperviseした。

Losses

4項をweight付きで加算した。

- CenterNet-style focal loss (weight=1.0) 、heatmap上で $\alpha=2$, $\beta=4$ 。
- L1 offset loss (weight=1.0) 、valid center cellだけ。
- series-level logitsからのBCE-with-logits classification loss (weight=1.0) 。
- vessel segmentation maskがあるときは2D auxiliary output上のAuxiliary Dice & BCE loss (weight=0.5) 、なければ0。slice-wise feature consistencyを促す。

Voxelwise mapからseries-level logitsへ

heatmap logits $H \in \mathbb{R}^{B \times 13 \times D \times H' \times W'}$ に対し：

- 13 classそれぞれのper-class series logitは (D, H', W') にわたるspatial max。
- Aneurysm Present global logitは13 class logitのmax。
- この14 logitsでBCE lossを計算し、meta-ensembling用にexportした。

Data pipeline & augmentations

- **Input:** precomputed .npy volume。 (D, H, W) を $(1, D, H, W)$ へexpandし、 $D=64$ 。
- **Label scaling:** current volume sizeへcoordinateをrescaleし、target grid用のstride mappingを適用。
- **Augmentation:** volumeごとに共通の2D affine warp (全sliceへ同じtransform) 。random rotation、scale、translation。vertical flipも使用可能だがdefaultではoff。
- **Normalization:** Albumentations normalizationからtensor変換。必要なら3-channelへrepeat。

Optimization & schedules

- AdamW (LR= $2e-4$ 、weight decay= $1e-5$) 、AMP mixed precision、grad clipping=1.0。
- defaultはCosineAnnealingLR ($T_{max}=\text{epochs}$, $\eta_{min}=1e-6$) 。StepLR / ReduceLROnPlateauもsupport。
- validation mean AUCでbest modelを選択。

Training configuration

- Epochs: 16
- Batch size: 4 (train) / 2 (val)
- accumulate: 2
- Workers: 16
- Strides: depth 1、lateral 16
- Gaussian σ : 0.2
- Backbone: EfficientNetV2-S、pretrained=True、defaultではfreezeしない

Cross-validation & metrics

training CSV splitによるK-fold protocolを使った。各epochでper-label ROC-AUC、mean AUC、presence-weighted AUCを計算した。foldごとにschedulerをstepし、mean AUCでbest checkpointを保存した。final summaryではfold別とaverage AUCを報告する。

Flayer without aux head	loc_macro_auc	cls_auc	combined_mean
fold0	0.7762	0.7747	0.7754
fold1	0.7676	0.8069	0.7873
fold2	0.7508	0.7892	0.7700
fold3	0.7553	0.7778	0.7666
fold4	0.7734	0.7758	0.7746
Average	0.7647	0.7848	0.7748

Flayer with aux head	loc_macro_auc	cls_auc	combined_mean
fold0	0.7779	0.7991	0.7885
fold1	0.7828	0.7987	0.7908
fold2	0.7601	0.8051	0.7826
fold3	0.7683	0.7826	0.7755
fold4	0.7586	0.7954	0.7770
Average	0.7695	0.7962	0.7829

Flayerは2D EfficientNetV2-Sのslice featureを軽量3D headで3D evidence mapへ変換し、CenterNet-style heatmap + offsetでsuperviseする。aggregated Aneurysm Present logitを含むrobustな14-logit series-level predictionを生成し、stacked meta-classifierへそのまま接続できる。

Meta-classifier

YOLO11m、YOLO11-EffV2s、EffV2s-3D-CenterNetのpredictionを統合するstacked ensembleを設計した。各 meta-classifierは全base modelのconcatenated predictionをinputとし、aneurysm presenceとlocation-specific detectionのfinal predictionを出す。全model・全locationのpredictionを考慮して特定locationのaneurysm presenceを評価し、ensemble全体の判断を捉えられる。

```
metadata = np.array([age, sex])
X = np.concatenate([np.array([yolo11m_cls_preds[fold_id]]), yolo11m_loc_preds[fold_id],
                        np.array([effv2s_cls_preds[fold_id]]), effv2s_loc_preds[fold_id],
                        flayer_fold_preds[fold_id], metadata], axis=0)

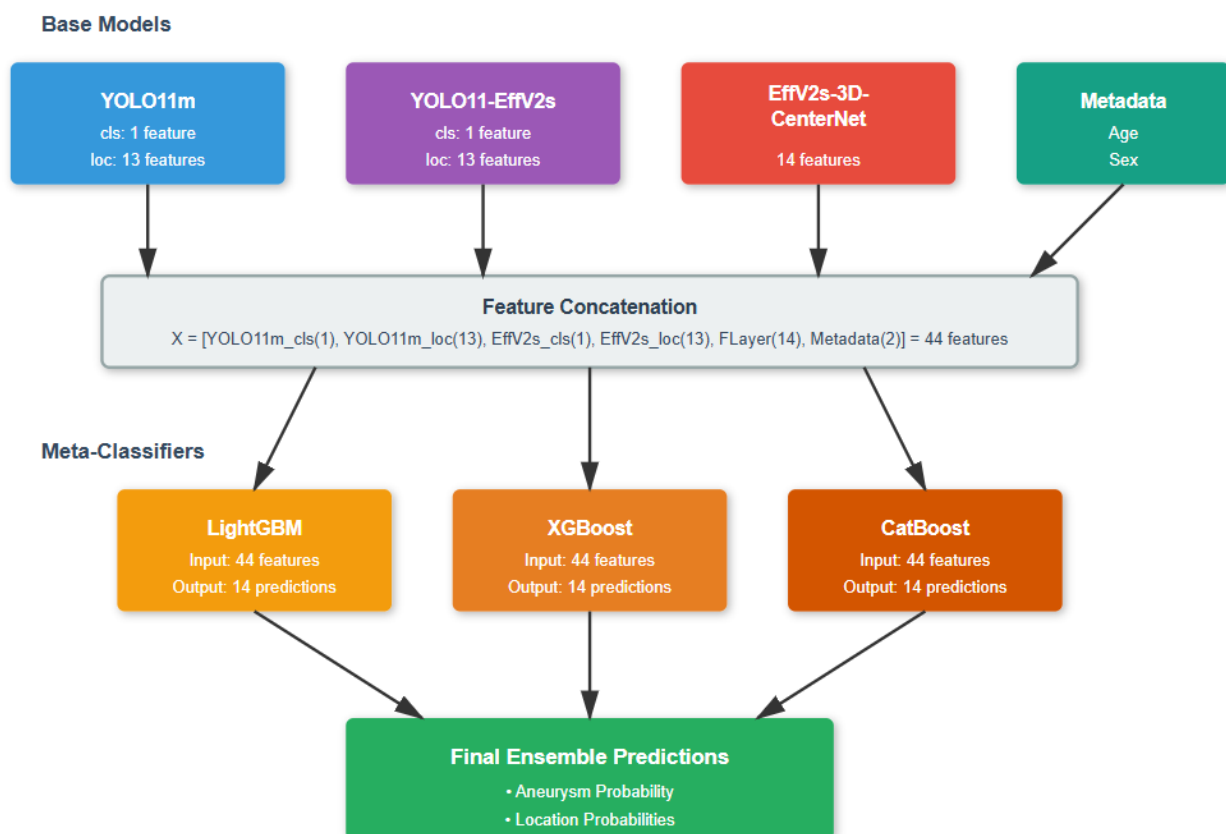
lgb_pred = predict_prob_lgb(X, fold_id)
xgb_pred = predict_prob_xgb(X, fold_id)
cat_pred = predict_prob_cat(X, fold_id)
```

2種類のensemble structureを試した。

- **Series:** single YOLO prediction + one Flayer predictionをensemble modelへ入力してfinal outputを得る。
(features, [() + flayer() + meta()])
- **Parallel:** 2 YOLO prediction + 1 Flayer predictionをparallelに処理し、共同でfinal predictionを得る。
(features, [yolo11m() + () + () + ()])

parallelなmultiple feature streamがseries configurationを超えるcomplementary informationを与えるか比較した。異なるYOLO backboneとFlayerのpredictionをparallelに組み合わせた方がensemble CV scoreを改善した。GBDTが各modelの全locationにおけるaneurysm presenceの認識を有効活用していることを示す。最終日の問題で5-fold submissionを完了できなかったため、最終選択結果は2 fold averageである。

Metric	Series	Parallel
5 folds CV (average)	0.84	0.852
5 folds CV (Nelder-Mead optimized)	0.841	0.858
Public score (average; 2 folds)	0.83	0.83082
Private score (average; 2 folds)	0.81	0.8230



TomのBYU approachにあるKeypoint-Based Dual-Graph Predictorも以前試し、ensemble CVは良好だった。しかしYOLO-based feature extractionとpreprocessingに時間がかかりすぎ、頻繁にtimeoutしたためfinal solutionには含めなかった。

Codebase & submission

- [Submission: 2x YOLO + Flayer + meta-classifier](#)
- [Kaggle models submission](#)
- [Meta-classifier training](#)
- [GitHub](#)

補足Q&A

Ujjwal Pandey (質問) : preprocessingでNIfTI volumeのreorientation/resamplingを試しましたか。pipelineでGBDTを使った具体的理由は何ですか。test setにはAge + Sex metadataがないと思いますが、単に利便性のためでしょうか。2D/2.5D solutionを選んだ動機は何ですか (thick/thin sliceの混在が理由ですか) 。pure end-to-end 3D solutionも試しましたか。

atom1231 (回答) : pure 3D solutionの最大の問題はcompute不足と長すぎるtraining timeでした。解法中のCenterNetはcompute/time制約を緩和するための妥協した、切り詰めた3D solutionです。

Tom (回答) : meta-classifierを使った理由は、あるclassを予測するとき、他classのpredictionをaggregateするとCVが大きく改善すると分かったためです。他classのpredicted probabilityも特定locationのaneurysmを識別する有用な手掛かりになると考えました。sample test dataにはAgeとSexがありましたが、submit後はmetadataの有無でLB scoreはほとんど変わらず、CVには明確な差がありました。そのためhidden test setの一部にも含まれると仮定し、AgeとSex featureを残しました。最初の1か月で3D classifierはoverfitしやすいと分かりました。各voxelをchannel dimensionで表すとoverfitしやすかったためです。一方、aneurysmを示し得る不規則なstructure changeの検出にはdepth contextが重要で、slice-wise predictionも期待どおりではありませんでした。各pixelを3つのlocal adjacent depthでchannel表現する方法が他の方法を上回りました。この方法はYOLOが小さなdepth structure changeに基づいてanomaly locationを捉えるのに有効で、single sliceをRGBへrepeatするYOLOよりCVが良好でした。